



## مشروع مادة البرمجة المتوازية (High-Performance E-Commerce Backend Engine)

الطلاب المشاركون:

طارق حسام الصباغ التاجي

محمد معاذ محمد سامر سويد

احمد طلال سعود

محمد حمزة محمدرائد مقداد

محمد احمد عاشور

✍ يوثق هذا التقرير التطبيق المعماري للمتطلبات غير الوظيفية في نظام (Parallel Cart) المبني باستخدام (Spring Boot). يطبق المشروع جميع المتطلبات غير الوظيفية العشرة على مستوى البنية المعمارية ، مع توفر أدلة الإثبات في الشيفرة المصدرية، والاختبارات، والتقارير المولدة .

- تم اعتماد منهجية (قبل/بعد) (Before/After) لإثبات كفاءة الحلول، حيث يمثل مسار "قبل" السلوك بدون حماية (Baseline)، بينما يمثل مسار "بعد" التنفيذ الحالي للمشروع.

#### 1. حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

- التطبيق المعماري :تم استخدام أنوتيشن @Version في صنف Inventory لتطبيق القفل المتفائل (Optimistic Concurrency Control). وفي حال حدوث تضارب، يعتمد النظام على استراتيجية إعادة المحاولة مع تأخير عشوائي (Retry + Jitter) ، تليها ترقية إلى القفل التشاؤمي (Pessimistic Fallback).
- الشرح التقني :عند الشراء، يتم محاولة التحديث بشكل متفائل لتسريع الأداء وتجنب حظر الموارد. إذا اكتشف النظام تضارباً بالنسخ، يقوم بإعادة المحاولة حتى 3 مرات مع تأخير عشوائي (20-80 ملي ثانية) لتقليل موجات التضارب. إذا استمر التضارب، يتم قفل السطر في قاعدة البيانات قفلاً تشاؤمياً (PESSIMISTIC\_WRITE) لفرض التسلسل وحماية البيانات .
- المقارنة والتجارب

**قبل :تجاوز الحد المسموح للمخزون (Oversell) واحتمال تلف البيانات.**

**بعد : لا يوجد أي تجاوز للمخزون، ويتم التعامل مع التضاربات كإعادة محاولات مسيطر عليها أو إرجاع رمز خطأ (HTTP 409) بدلاً من تلف البيانات.**

```
error_type=IllegalStateException
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.608 s -- in com.parallelcart.CheckoutTransactionIntegrityTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.376 s
[INFO] Finished at: 2026-05-18T21:44:50+03:00
[INFO] -----
~/projects/farth-year/parallel-cart main +1 9s 09:44:50 PM
> ./mvnw -Dtest=CheckoutTransactionIntegrityTest test
21:45 18-May
```

- التطبيق المعماري: تم استخدام تجمعات خيوط مقيدة (Bounded Thread Pools) ، وتحديد سعة لطوابير الانتظار، إضافة إلى حارس يعتمد على خوارزمية الإشارة (Semaphore Fail-fast Guard)
- الشرح التقني: توجد الحماية على طبقتين؛ الأولى تحمي موارد التنفيذ عبر تقييد الخيوط لمنع الاستهلاك المفرط للذاكرة. الثانية تحمي نقطة الاختناق (Checkout) عبر (Semaphore) يضع سقفاً للعمليات المتزامنة، وإذا امتلأ يتم الرفض السريع وإرجاع (HTTP 503) ، مما يحمي الخادم من طوابير الانتظار اللانهائية .
- المقارنة والتجارب:

قبل: ارتفاع في زمن الاستجابة (Latency) واحتمال حدوث مجاعة للخيوط. (Thread Starvation).

بعد: تحكم بالضغط عبر إرجاع استجابات 503، وبقاء عملية المعالجة مستقرة دون انهيار .

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CX, and | <pre> rSaturationTest in 2.25 seconds (process running for 3.039) [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.777 s -- in com.parallelcart.api.controller.CartControllerSaturationTest [INFO] Results: [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 [INFO] ----- [INFO] BUILD SUCCESS [INFO] ----- [INFO] Total time: 4.545 s [INFO] Finished at: 2026-05-18T21:46:41+03:00 [INFO] ----- </pre> |
| and     | <pre> ~/projects/forth-year/parallel-cart main 6s 09:46:41 PM &gt; ./mvnw -Dtest=CartControllerSaturationTest test </pre>                                                                                                                                                                                                                                                                                                               |
|         | 21:46 18-May                                                                                                                                                                                                                                                                                                                                                                                                                            |

```
[config-check]
$ rg -n "app.executors|spring.kafka.listener.concurrency" src/main/resources/application-local.yml src/main/java/com/parallelcart/config/ResourceManagementConfig.java
src/main/resources/application-local.yml
36:      core-pool-size: 4
37:      max-pool-size: 8
38:      queue-capacity: 100
41:      pool-size: 2

src/main/java/com/parallelcart/config/ResourceManagementConfig.java
15:      @Value("${app.executors.async.core-pool-size:4}") int corePoolSize,
16:      @Value("${app.executors.async.max-pool-size:8}") int maxPoolSize,
17:      @Value("${app.executors.async.queue-capacity:100}") int queueCapacity,
18:      @Value("${app.executors.async.keep-alive-seconds:60}") int keepAliveSeconds) {
32:      @Value("${app.executors.scheduler.pool-size:2}") int poolSize) {

~/projects/furth-year/parallel-cart main 09:47:15 PM
21:47 18-May
```

```
[runtime-503-attempt]
Firing concurrent checkout requests with max-concurrent=1
HTTP code counts:
    1 200
   39 503

~/projects/furth-year/parallel-cart main 40s 09:48:09 PM
21:48 18-May
```

### 3. المعالجة غير المتزامنة (Asynchronous Queues)

- التطبيق المعماري: تم استخدام نمط صندوق الصادر المدمج بالمعاملات (Transactional Outbox)، مجدول للنشر، ومستهلكين عبر منصة (Kafka) مع طابور للرسائل الميتة (DLQ)
- الشرح التقني: تقوم عملية الدفع بكتابة بيانات العمل وحدث الـ Outbox في نفس المعاملة لمنع عدم الاتساق (Dual-write inconsistency) لاحقاً، يقوم المجدول بنشر الأحداث إلى Kafka. هذا يقصر مسار عملية الدفع ويرفع الموثوقية في حال حدوث مشاكل عابرة في الاتصال. المستهلكون مصممون ليكونوا عديمي التأثير بالتكرار. (Idempotent).
- المقارنة والتجارب:

**قبل:** تنفيذ المهام الجانبية ضمن نفس المسار يؤدي إلى زمن استجابة أعلى لعملية الدفع وزيادة احتمالية فشل الطلب.

**بعد:** انخفاض زمن الاستجابة للدفع، وإمكانية إتمام الشراء بنجاح بينما تستمر الوظائف الخلفية بمعالجة المهام الجانبية.

```
[outbox-row-initial]
$ docker compose exec -T postgres psql -U parallel_cart -d parallel_cart -c "select id, event_type, aggregate_id,
status, publish_attempts, published_at, left(coalesce(last_error, ''), 120) as last_error from outbox_events whe
re aggregate_id = 1 order by id desc;"
 id | event_type | aggregate_id | status | publish_attempts | published_at | last_error
-----+-----+-----+-----+-----+-----+-----
  1 | ORDER_CREATED |          1 | PENDING |          0 |              |
(1 row)
```

```
$ wait_for 'invoice row' (up to 120s)
[ok] invoice row count=1
$ wait_for 'notification row' (up to 120s)
[ok] notification row count=1
$ wait_for 'published outbox row' (up to 120s)
[ok] published outbox row count=1

[invoice-row]
```

#### 4. معالجة البيانات الضخمة على دفعات (Batch Processing)

- التطبيق المعماري: حلقة يومية لجرد المبيعات على شكل دفعات (Chunks) مع نقاط تحقق للاستئناف (Checkpoint Resume) ومهمة مجدولة .
- الشرح التقني: يتم استرداد الطلبات المدفوعة تصاعدياً على شكل دفعات محددة الحجم. يتم تحديث الإجماليات ضمن نقطة التحقق (Checkpoint). هذا التصميم يحمي الذاكرة ويسمح باستئناف العمليات ويدعم قواعد البيانات الضخمة .
- المقارنة والتجارب:

قبل: سحب البيانات ككتلة واحدة يؤدي لضغط أعلى على الذاكرة وفقدان التسامح مع الأخطاء (Poor fault tolerance) عند الانقطاع .

بعد: استقرار في الذاكرة، واستئناف آمن من آخر طلب تمت معالجته في حال التوقف.

```
~/projects/forth-year/parallel-cart main
└─> ./mvnw -Dtest=DailySalesAggregationServiceTest test
└─> ./mvnw -Dtest=DailySalesAggregationCheckpointResumeTest test
└─> ./scripts/verify_phase5.sh t1
ec... ./scripts/verify_phase5.sh t2
it 4: htop
```

#### 5. توزيع الأحمال (Load Distribution)

- التطبيق المعماري: الاعتماد على خادم Nginx وتطبيق استراتيجية least\_conn للربط بين نسختين من التطبيق (app-1) و (app-2) مع إثبات التوزيع عبر ترويسات الاستجابة .
- الشرح التقني: يقوم Nginx بتوجيه كل طلب وارد إلى الخادم الخلفي صاحب العدد الأقل من الاتصالات النشطة، مما يحسن التعامل مع الطلبات ذات أزمنة التنفيذ المتفاوتة مقارنة بالتوزيع الدائري البسيط (Round-robin).



قبل: تطبيق وحيد يعمل كخلفية مما يرفع خطر الاختناق ويقلل التوسعية الأفقية.

بعد: ظهور الطلبات موزعة على الخوادم، ومرونة أعلى في التعامل مع الاتصالات.

```
=> [app-1] resolving provenance for metadata file
[+] Running 12/12
✓ app-1 Built
✓ app-2 Built
✓ Volume "parallel-cart_redis_data" Created
✓ Volume "parallel-cart_kafka_data" Created
✓ Volume "parallel-cart_postgres_data" Created
✓ Container parallel-cart-zookeeper Started
✓ Container parallel-cart-postgres Healthy
✓ Container parallel-cart-redis Healthy
✓ Container parallel-cart-kafka Healthy
✓ Container parallel-cart-app-2 Started
✓ Container parallel-cart-app-1 Started
✓ Container parallel-cart-nginx Running
[ok] load balancer reachable: http://localhost:8088/actuator/
```

```
172.21.0.6:8080
172.21.0.7:8080
172.21.0.6:8080

Counts:
  25 172.21.0.6:8080
  35 172.21.0.7:8080

[ok] P5-T3 verified: traffic reached multiple app instances

~/projects/forth-year/parallel-cart main
```